Calcul parallèle - 4OS02

Optimisation Par Colonie De Fourmi D'un Chemin Sur Un Paysage Fractal

Paul-Henri Djoko
Henry Royer
Théotime Galmiche

2026

Table des matières

Introduction **2**

1 Preliminaires de Parallelisation **2**
1.1 Analyse du Code existant 2
1.1.1 Machine d'analyse 2
1.1.2 Profiling du Code 2
1.2 Vectorisation 3
1.3 OpenMP 4

2 Première façon **4**

3 Seconde façon **4**

Introduction

Les algorithmes inspirés du comportement collectif d'insectes sociaux, comme l'optimisation par colonie de fourmis (ACO), permettent de résoudre certains problèmes d'optimisation de manière distribuée.

Dans ce projet, nous simulons une colonie de fourmis évoluant sur une grille représentant un terrain fractal afin de trouver un chemin entre une fourmilière et une source de nourriture. L'objectif est également d'étudier les performances du programme à travers la vectorisation et la parallélisation du code.

1 Preliminaires de Parallelisation

1.1 Analyse du Code existant

1.1.1 Machine d'analyse

L'analyse qui suit a ete faite sur une petite machine dont la taille des caches est comme suit :

```
Caches (sum of all):
  L1d:                        64 KiB (2 instances)
  L1i:                        64 KiB (2 instances)
  L2:                         512 KiB (2 instances)
  L3:                         4 MiB (1 instance)
```

Cette petite taille de cache influera sur les resultats de la sous-section suivante.

1.1.2 Profiling du Code

Sur la map ci-dessus, en jaune se trouvent tous les appels de fonctions issus de la bibliothèque std : :vector ou std : :array. On comprend donc l'importance de la vectorisation, qui permettra de gagner un facteur 10 sur la performance comme on l'a souvent vu en TP.

Nous suspectons que la petite taille des caches L1 sur cette machine rendent l'étape d'évaporation des phéromones plus longue, car le calcul étant vraiment trivial, le système requiert tout simplement beaucoup de temps pour charger les valeurs dans le cache, petit à petit, en attendant toujours ces transferts de données, le temps effectif de travail du cpu est donc très faible. Ceci est vérifié par le profiling du programme sur un ordinateur plus performant

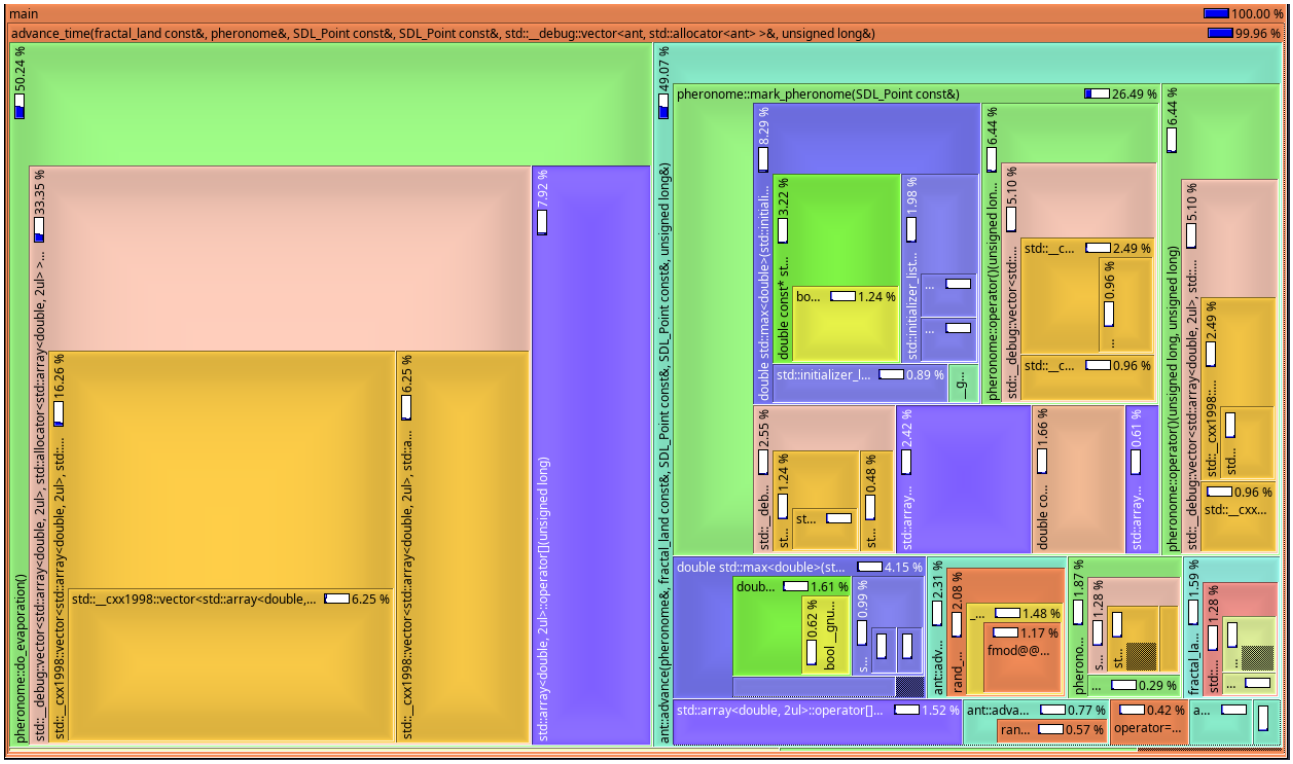

FIGURE 1 – Callee map de valgrind/Kcachegrind

qui voit une séparation 70-30 entre l'avancement des fourmis et la phase d'évaporation par rapport au 50-50 que l'on observe ici.

La taille de ces deux zones principales nous force aussi à les paralléliser toutes deux, car cette division 50-50 nous donne un speedup théorique maximum de 2 si seulement l'une d'entre elle était parallélisée.

1.2 Vectorisation

La vectorisation va nous permettre d'utiliser la parallélisation OpenMP. En effet, avec la vectorisation, on a un vector de position_t pour les positions, un vector de int pour les états et un vector de size_t pour les seeds. Chaque fourmi est identifiée par un indice et chaque thread i ne manipule que son indice i dans chaque tableau.
Par conséquent on a aussi modifieé les méthodes advance de la classe ant et rerender de la classe Rerender afin qu'elle prenne en argument une position, une graine et un état et non une "ant". on a aussi modifieé la fonction advance time en conséquence.
Cette vectorisation n'améliore pas les performances. Arrivé à la 1000ème itération on obtient que chaque itération prend 3.47ms calculé avec le module <chrono>. 82.2% du temps est pris par la mise à jour des fourmis, 17.4% par l'évaporation et 0.4% par l'update.

1.3 OpenMP

Ainsi, nous avons conclu que la boucle qu'il fallait paralléliser était celle dans laquelle était la fonction advance. On note une nette amélioration des performances.
Avec cette parallélisation OpenMP, le pourcentage du temps passé à mettre à jour les fourmis diminue.

nb threads	temps/iteration (ms)	speed up	% fourmis	% evap	% update
2	2.61	1.33	67.5	31.9	0.6
4	2.37	1.46	51.8	47.8	0.4
8	2.21	1.57	64.3	35.0	0.7
16	2.27	1.52	65.3	34.0	0.7

2 Première façon

Cette méthode consiste à avoir tout l'environnement dans chaque processus et se partager les fourmis entre processus. Cette méthode peut sembler idéale mais la mise à jour des phéromones fait qu'une grande quantité d'information (toute la carte) est échangée entre chaque processus à chaque itération.
Ainsi plus on utilise de coeur moins les performances sont bonnes et plus les parties update et evaporation prennent de la place.

nb coeurs	temps/iteration (ms)	speed up	% fourmis	% evap	% update
2	4.93	1.33	34.9	59.1	6.0
4	29.47	1.46	4.9	53.3	41.8
8	52.72	0.07	1.5	46.7	51.8

3 Seconde façon

Dans cette approche, la carte est découpée en sous-grilles et chaque processus s'occupe d'une portion de la grille uniquement. Il gère donc les cellules et les fourmis qui se trouvent dans cette portion.

Lorsqu'une fourmi se déplace vers le bord d'une sous-grille, elle atteint une ligne/colonne partagée avec le processus qui gère la portion d'à côté. Si la fourmi reste dans la zone où elle était, rien ne se passe, mais si elle traverse effectivement dans l'autre portion, on envoie ses informations (position, état, chargée ou non) au processus qui gère la portion entrante et qui prendra ensuite en charge la suite de son déplacement.

De plus, la synchronisation des bords des sous-cartes est importante. Il faut ainsi envoyer les valeurs des phéromones à chaque itération au voisin qui partage le bord en question. Cela assure que le calcul reste cohérent.

Ainsi, la stratégie peut se résumer ainsi : pour chaque pas de temps et pour chaque processus, la boucle principale consiste en :

— le déplacement de toutes les fourmis locales selon les règles de l'ACO (exploration, suivi des phéromones, et chargé/non chargé).
— la mise à jour des phéromones locales de la sous-carte en utilisant les valeurs des voisins.
— l'envoi et la réception des fourmis qui franchissent les frontières vers les processus voisins.
— la synchronisation des phéromones sur les bords avec les sous-cartes voisines pour garantir la cohérence des calculs.